Task ‘Unit 5 Design Patterns III - Behavioural Patterns’ Unit 5

Task Collaborative Discussion 2

Refactoring with the Strategy Pattern

Problems in Current Implementation

The PaymentProcessor class is violating the Open/Closed Principle by adding new payment methods that require modifying existing code. Additionally, the if-elif chain creates tight coupling, poor scalability and reduced testability.

(Freeman and Robson, 2020).

How Strategy Pattern Improves the Code

The Strategy Pattern encapsulates each payment algorithm into its own class, allowing PaymentProcessor to delegate to interchangeable strategies without knowing their internals. New methods are added by creating new classes, not editing existing ones.

Refactored Code

```python
from abc import ABC, abstractmethod

# Step 1: Strategy Interface

class PaymentStrategy(ABC):

    @abstractmethod

    def pay(self, amount: float) -> None:

        pass
```

`` ``` ``

Step 2: Concrete Strategies

```python
class CreditCardPayment(PaymentStrategy):
    def pay(self, amount: float) -> None:
        print(f"Processing credit card payment of ${amount}")


class PayPalPayment(PaymentStrategy):
    def pay(self, amount: float) -> None:
        print(f"Processing PayPal payment of ${amount}")


class BankTransferPayment(PaymentStrategy):
    def pay(self, amount: float) -> None:
        print(f"Processing bank transfer of ${amount}")
```

Step 3: Refactored Context Class

```python
```

```
class PaymentProcessor:
    def __init__(self, strategy: PaymentStrategy) -> None:
        self._strategy = strategy

    def process_payment(self, amount: float) -> None:
        self._strategy.pay(amount)
```

Step 4: Usage

```python
processor = PaymentProcessor(CreditCardPayment())
processor.process_payment(100)

processor = PaymentProcessor(PayPalPayment())
processor.process_payment(200)
```

Benefits in This Scenario

Open/Closed Principle Adding CryptoPayment requires only a new class, zero changes to PaymentProcessor.

The processor depends on the abstraction (PaymentStrategy), not concrete implementations, which supports the loose coupling cocept.

Each strategy is independently unit-testable, which improves testability.

The logic is cleanly separated per responsibility what improves readability.

(Tsybulka, 2024)

References

- Freeman, E. & Robson, E. (2020) Head First Design Patterns: *Building Extensible and Maintainable Object-Oriented Software*. Second edition. O’Reilly.. Available at: https://essex.primo.exlibrisgroup.com/permalink/44UOES_INST/s7iv0e/cdi_askewsholts_vlebooks_9781492077978 [Accessed: 19 May 2026].
- Tsybulka, K., (2024). Enhancing code quality through automated refactoring techniques (Doctoral dissertation, ETSI_Informatica). [Accessed: 19 May 2026].

Use of Digital Tools

This document has been produced exclusively for educational purposes. Referenced content, copyrights, and trademarks, remain the sole property of the respective owner. ChatGPT 5.5 was used for some proofreading and clarity as well as literature exploration, language refinement, coding and programming techniques research. All academic writing, analysis, argumentation, and conclusions represent the original work of the author. AI tools were used for exploration, same way as academic literature exploration. No AI-generated content or code was directly copied into the final submission. Any use of content mentioned in this document is properly referenced to its original author and was used responsibly to ensure integrity and originality. All academic writing, analysis, argumentation, and conclusions represent my original work. I have used Artificial Intelligence responsibly and ethically, in accordance with the assignment brief. All AI use has been declared and evidenced.